

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
федеральное государственное автономное образовательное учреждение высшего образования
«САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
АЭРОКОСМИЧЕСКОГО ПРИБОРОСТРОЕНИЯ»

Кафедра информационных систем и технологий (№42)

ОТЧЁТ ПО ПРАКТИКЕ
ЗАЩИЩЁН С ОЦЕНКОЙ

РУКОВОДИТЕЛЬ

старший преподаватель

должность, уч. степень, звание

Отлично



подпись, дата

С.Ю. Гуков

инициалы, фамилия

ОТЧЁТ ПО ПРАКТИКЕ

вид практики учебная

тип практики

на тему индивидуального задания

Реализация компьютерной игры «Blackjack» на языке (на движке) «Unity»

выполнен Буренковым Григорием Владимировичем

фамилия, имя, отчество обучающегося в творительном падеже

по направлению подготовки

09.03.02

код

Информационные системы и технологии

наименование направления

наименование направления

направленности

09.03.02

код

Информационные технологии в бизнесе

наименование направленности

наименование направленности

Обучающийся группы №

4321

номер

29.05.2024

подпись, дата



Г.В. Буренков

инициалы, фамилия

Санкт-Петербург 2024

ИНДИВИДУАЛЬНОЕ ЗАДАНИЕ

на прохождение учебной практики обучающегося направления подготовки/ специальности
09.03.02 Информационные системы и технологии

1. Фамилия, имя, отчество обучающегося: Буренков Григорий Владимирович

2. Группа: 4321

3. Тема индивидуального задания:

Реализация компьютерной игры «Карточной» на языке (на движке) «Unity»

4. Исходные данные:

Visual Studio Code, Unity Hub; вариант задания (1).

5. Содержание отчетной документации:

5.1. индивидуальное задание;

5.2. отчет, включающий в себя:

– титульный лист;

– материалы о выполнении индивидуального задания (содержание определяется кафедрой);

– выводы по результатам практики;

– список использованных источников.

5.3. отзыв руководителя от профильной организации (при прохождении практики в профильной организации).

6. Срок представления отчета на кафедру: «29» мая 2024 г.

Руководитель практики
старший преподаватель
должность, уч. степень, звание

29.05.2024
подпись, дата

С.Ю. Гуков
инициалы, фамилия

Задание принял к исполнению:
Обучающийся

05.02.2024
дата

Г.В. Буренков
подпись

Г.В. Буренков
инициалы, фамилия

Санкт-Петербург 2024

СОДЕРЖАНИЕ

Введение	4
1 Индивидуальное задание	5
2 Тема и описание игры	7
2.1 Основные правила игры	7
3 Пользовательская документация.....	8
4 Техническая документация.....	10
4.1 Блок-схема	10
4.2 Описание взаимодействия объектов в Unity	10
4.3 Описание файлов в проекте	11
4.4 Описание основных функций	12
5 Тестирование программы	14
6 Демонстрация механики игры.....	15
Заключение.....	22
Список использованных источников.....	23
Приложение. Исходный код программы.....	24

ВВЕДЕНИЕ

Разработка компьютерных игр является одной из наиболее динамично развивающихся отраслей информационных технологий. Игры служат не только для развлечения, но и для обучения, тренировки когнитивных способностей и даже для научных исследований. Цель данного проекта — разработка компьютерной карточной игры Blackjack на платформе Unity.

Blackjack является одной из самых популярных карточных игр в мире, требующей стратегического мышления и навыков принятия решений. Проект представляет собой комплексную разработку, включающую как технические аспекты, так и элементы дизайна. В результате выполнения работы была создана полноценная игра, которая может быть использована как для развлечения, так и для тренировки стратегического мышления.

В ходе реализации проекта особое внимание было уделено созданию интуитивно понятного интерфейса и проработке алгоритмов, обеспечивающих реалистичное поведение игры и честные условия для игроков.

Цель данного проекта — разработка компьютерной карточной игры Blackjack на платформе Unity. Blackjack является одной из самых популярных карточных игр в мире, требующей стратегического мышления и навыков принятия решений.

1 Индивидуальное задание

Целью учебной практики является разработка простейшей игры на языке программирования C# с использованием графического интерфейса и движка Unity. В разработанном приложении должен быть реализован весь функционал, присущий выбранному жанру или игре.

Для реализации выбранной цели был выбран жанр карточных игр, точнее игра Blackjack с элементами правил 21 и 21+3.

Задачи: изучить правила игры Blackjack, 21 и 21+3, чтобы выявить оптимальные правила, сохраняя главные аспекты игр и интерес соревноваться с другими игроками. В том числе обязательными задачами являются:

- создание плана игры для ускорения разработки,
- изучение основ работы с платформой Unity (Ознакомление с интерфейсом, изучение основ C# программирования в контексте Unity, освоение основных компонентов и их взаимодействия),
- разработка игрового интерфейса (создание сцен для главного меню и игрового процесса, разработка графического интерфейса пользователя (GUI), настройка кнопок и других элементов управления),
- реализация игровой логики (программирование логики раздачи карт, реализация подсчета очков, программирование игровых действий: удар (Hit) и остановка (Stand), обработка ставок и расчет выигрышей/проигрышей),
- разработка и интеграция анимаций (создание анимаций для карт, интеграция анимаций в игровой процесс, реализация анимации перехода между сценами),
- подготовка пользовательской документации (описание интерфейса пользователя, инструкции по использованию игры),
- тестирование и отладка программы (проведение функционального тестирования, исправление ошибок и багов, выявленных в процессе тестирования, оптимизация производительности игры),
- поиск готовых Sprites, при отсутствии выделить время на разработку,

- изучение основ пользования Git для облачного хранения проекта и возможностью делиться им с коллегами,
- размещение проекта на GitHub (создание репозитория на GitHub, загрузка всех файлов проекта в репозиторий, оформление файла README.md с описанием проекта и ссылкой на видео).

2 Тема и описание игры

Игра Blackjack представляет собой карточную игру, целью которой является набрать сумму очков, как можно ближе к 21, но не превышая эту цифру. Игроки соревнуются против дилера, стремясь собрать более высокую комбинацию карт, чем у дилера, но не больше 21 очка. В большинстве случаев, игроки играют по стандартным правилам, придуманными в 15 веке, но в 21 веке начали придумывать аналоги игры с “турбо-правилами”. Такие правила включают множество вариантов игры: относительно руки игрока, комбинации на руке игрока, комбинации руки игрока и дилера. В мною представленной игре используются правила стандартного Blackjack и его турбо-версии.

2.1 Основные правила игры

Игра начинается с того, что каждому игроку и дилеру раздаются по две карты. Одна из карт дилера скрыта до конца игры.

Стоимость карт от 2 до 10 соответствует их номиналу. Карты "валет", "дама" и "король" оцениваются в 10 очков. Туз может стоить либо 1, либо 11 очков, в зависимости от того, что выгоднее для игрока.

После раздачи карт игрок может выбрать "ударить" (получить еще одну карту) или "остановиться" (сохранить текущую руку). Игрок может брать карты до тех пор, пока не решит остановиться или пока сумма его очков не превысит 21 (что означает проигрыш).

Увеличивать депозит можно в любой момент игры, не в зависимости от выбора действия, потому что до последнего игрок не может быть уверен, что выиграет он или нет из-за скрытой карты дилера.

Когда игроки завершают свои ходы, дилер открывает свою скрытую карту и обязан брать дополнительные карты до тех пор, пока сумма его очков не достигнет 17 или более.

Если сумма очков игрока превышает 21, игрок проигрывает. Если дилер превышает 21, игрок выигрывает. В остальных случаях выигрывает тот, у кого сумма очков ближе к 21.

3 Пользовательская документация

Этот проект представляет собой карточную игру, разработанную для учебных целей. В этой документации вы найдете информацию о том, как установить, настроить и использовать проект

Для использования данного приложения ниже описаны системные требования:

- операционная система: Windows, macOS или Linux,
- Unity Hub версии 3.8.0 или выше,
- язык программирования C# версии 6.0 или выше,
- Git (GitHub).

Шаги по установке. Склонируйте репозиторий. На рисунке 1 представлена консольная команда для клонирования репозитория игры на компьютер.

```
git clone https://github.com/skv0r/CardGameGUAP.git
```

Рисунок 1 – Консольная команда для клонирования репозитория игры

Открой в Unity Hub скачанную папку. На рисунке 2 представлен процесс добавления игры в Projects в Unity Hub для запуска проекта.

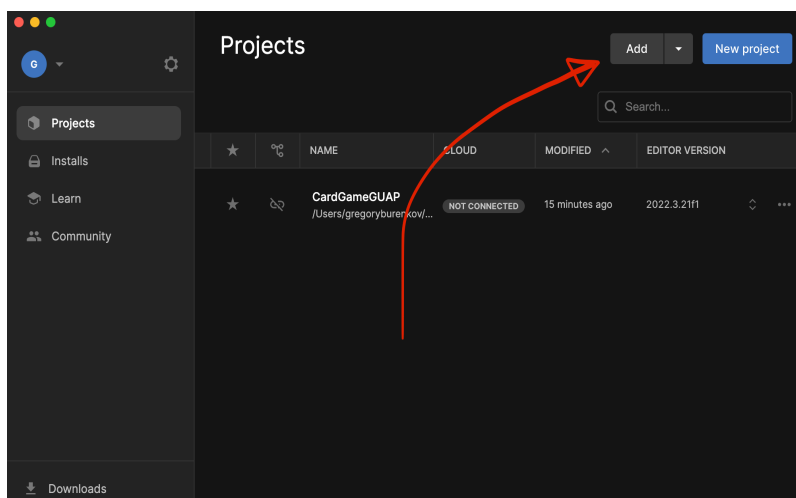


Рисунок 2 – Процесс добавления игры

Запустите нажатием на добавленный проект игру. Выберите оптимальное разрешение для вашей системы. Запустите игру нажатием на кнопку Play в верхней части экрана.

Основные функции и возможности игры:

- в главном меню есть возможность настроить громкость воспроизведения фоновой музыки,
- по нажатию на вкладку Rules, игрока перенесет на текстовую доску, где описаны правила карточной игры, последовательность ходов, всевозможные действия дилера и игрока,
- после прочтения правил нажмите на кнопку Play, чтобы перейти на игровое поле, представленное зеленым тканевым полем с таблицей счета, денег и размером руки дилера и игрока,
- для начала игры выберите желаемую ставку нажатием на кнопку Bet (минимальная стоимость одного круга 20\$, что автоматически ставиться при запуске игры). После запуска исчезают ненужные кнопки, что упрощает пользование игрой новому игроку,
- в случае проигрыша всех денег, игроку предложат начать игру заново/выйти из игры в главное меню.

4 Техническая документация

4.1 Блок-схема

На рисунке 3 представлена блок-схема работы игры в среде Unity.

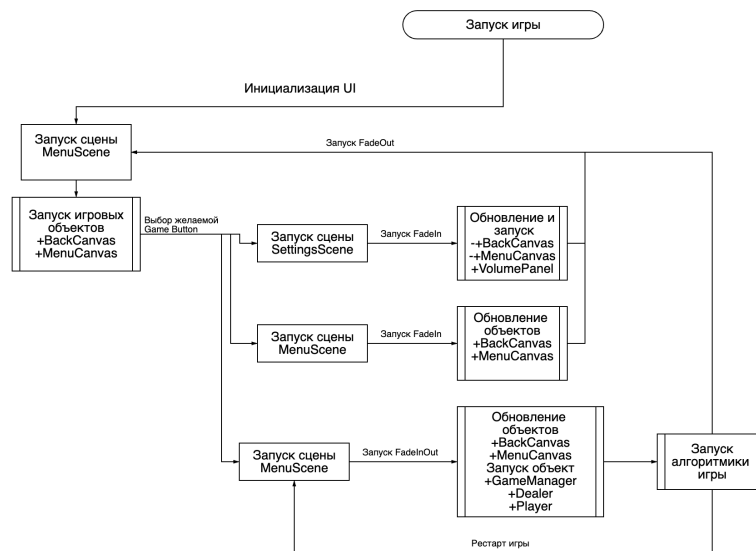


Рисунок 3 – Блок-схема работы игры

4.2 Описание взаимодействия объектов в Unity

В игровой среде Unity взаимодействие игрока с игрой реализовано с помощью функций сцен. Первично при запуске игры движок загружает нулевую сцену “MenuScene”, в данной сцене объекты разделены на два Canvas (BackCanvas и MenuCanvas). Первый объект имеет подобъекты, которые отвечают за GUI. Второй объект включает в себя UI, в том числе: анимации, игровые кнопки, текстовые элементы интерфейса, картинки и методы подгрузки различных Sprites.

При переходе между сценами, путем взаимодействия с игровыми кнопками, каждый новый запуск других сцен обновляет BackCanvas и MenuCanvas.

Исключением является основной игровой файл “GameScene”, для правильной работы проекта был добавлен локальный “движок” управления сценой – “GameManager”. Объект находится на самом верхнем уровне

иерархии, он является проводником между скриптами и игровыми объектами, что позволяет управлять состоянием объектов в коде.

4.3 Описание файлов в проекте

Таблица 1 - Описание основных файлов в игре.

Название файла	Описание
CardScript.cs	Этот скрипт управляет отдельными картами в игре, включая их значения, отображение и сброс.
DeckScript.cs	Этот скрипт управляет колодой карт в игре, включая их значения, перемешивание, раздачу и отображение рубашки карты.
PlayerScript.cs	Этот скрипт управляет действиями игрока/дилера в игре, включая начало раздачи, получение карт, проверку туза, корректировку денег и сброс руки.
GameManager.cs	Этот скрипт управляет основной логикой игры, включая действия игрока и дилера, обработку ставок и завершение раундов. Также является связующим звеном между игровыми объектами и кодом.
MusicScript.cs	Этот скрипт управляет воспроизведением фоновой музыки в игре.
SceneRestart.cs	Этот скрипт отвечает за перезагрузку текущей сцены.
SceneSwitcher.cs	Этот скрипт управляет переключением между сценами в игре.
VolumeVolume.cs	Этот скрипт управляет общим уровнем громкости в игре.

4.4 Описание основных функций

Таблица 2 - Описание основных функций в игре

Название функции	Принимаемый тип данных	Описание функции
PlayerScript.cs		
StartHand()	void	Раздает две карты и инициализирует руку игрока
GetCard(card)	GameObject	Добавляет карту в руку игрока и обновляет его очки
AceCheck()	void	Корректирует значение туза, чтобы избежать превышения 21 очка
AdjustMoney()	void	Изменяет баланс игрока на указанную сумму
ResetHand()	void	Очищает руку и обнуляет очки игрока
DeckScript.cs		
GetCardValues()	void	Возвращает массив значений всех карт в колоде
Shuffle()	void	Перемешивает карты в случайном порядке
DealCard()	void	Возвращает первую карту из колоды и удаляет её из списка
GetCardBack()	void	Возвращает спрайт карты
CardScript.cs		
ResetCard()	void	Сбрасывает значение карты и визуальное отображение
SetSprite(new Sprite)	Sprite	Устанавливает изображение карты, обновляя его новым спрайтом

Продолжение таблицы 2

Название функции	Принимаемый тип данных	Описание функции
GetSpriteName()	void	Возвращает имя текущего изображения карты
SetValue(newValue)	int	Устанавливает новое значение карты
GetValueOfCard()	void	Возвращает текущее значение карты
GameManager.cs		
DealClicked()	void	Начинает новый раунд игры, раздавая карты игроку и дилеру
HitClicked()	void	Добавляет карту игроку и проверяет, не превысил ли он 21 очко
StandClicked()	void	Завершает ход игрока и начинает ход дилера
HitDealer()	void	Добавляет карту дилеру и проверяет, не превысил ли он 21 очко. Продолжается до тех пор, пока дилер не наберет минимум 17 очков
RoundOver()	void	Проверяет итог игры и определяет победителя. Сравнивает очки игрока и дилера и обновляет состояние игры
BetClicked()	void	Обновляет ставку игрока и запускает новый раунд, если ставка валидна

5 Тестирование программы

Для тестирования сделаем UnitTest каждого скрипта, отвечающего за логику выдачи функционирования с объектами карт. На рисунке 4 представлено успешный UnitTest карточных функций.

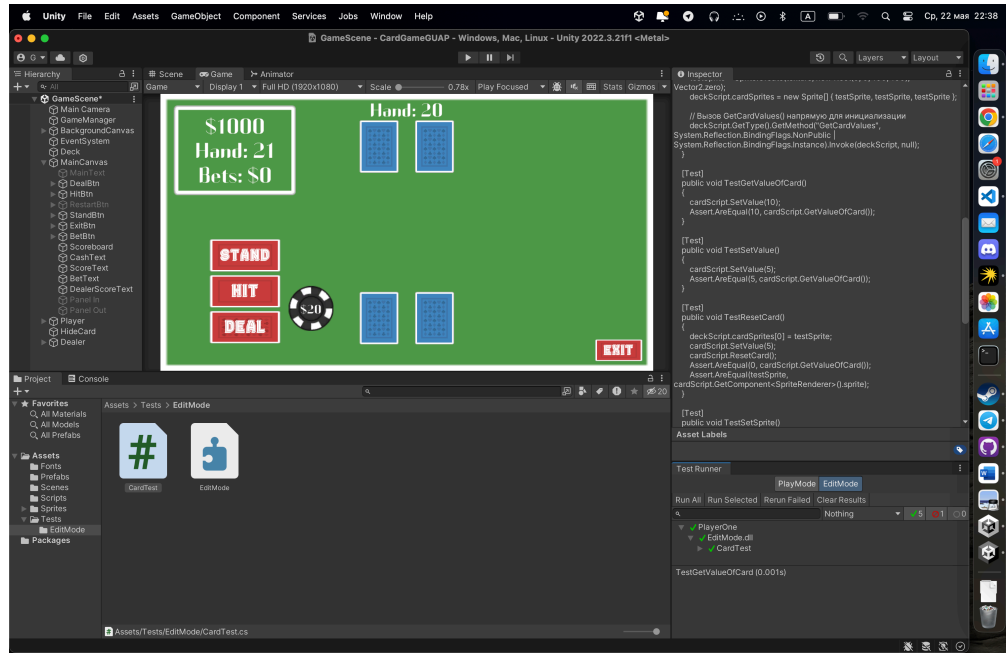


Рисунок 4 - Успешный UnitTest карточных функций

На рисунке 5 представлено успешный UnitTest GameManager функций.

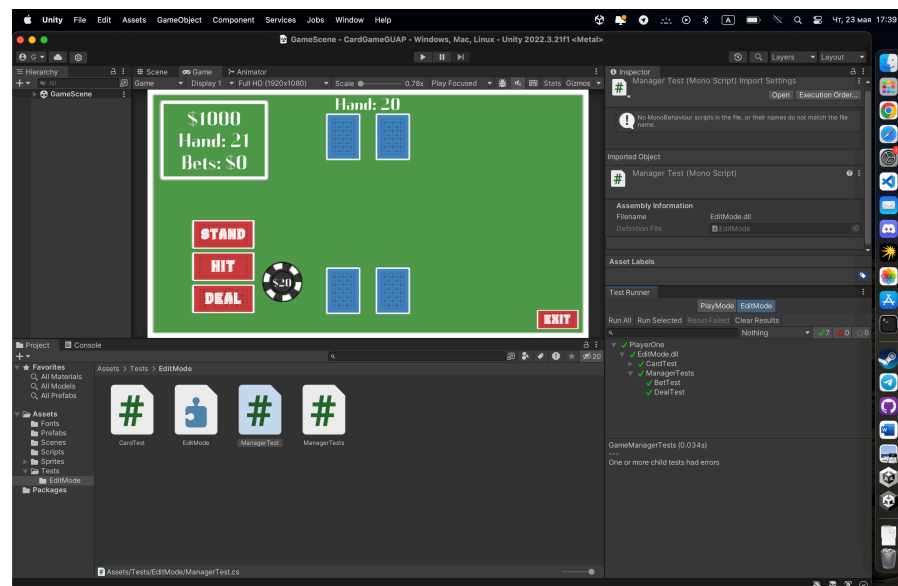


Рисунок 5 - Успешный UnitTest GameManager функций

6 Демонстрация механики игры

Ссылка на видео механик игры - <https://youtu.be/YoAv0CP3xms>. На рисунке 6 представлены фото основные механики игры.

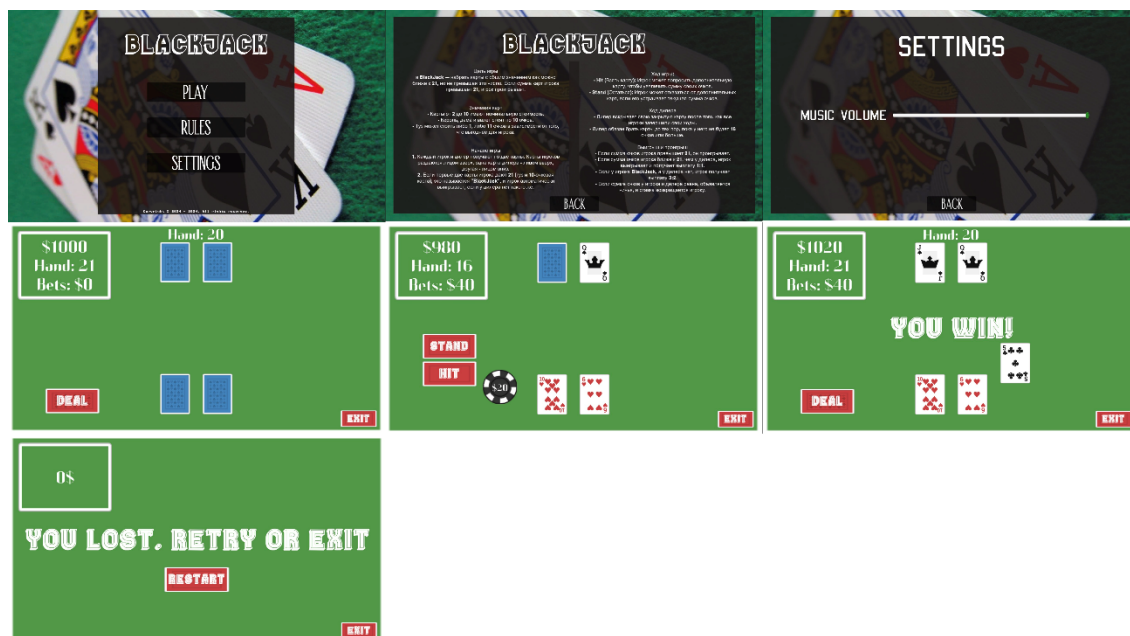


Рисунок 6 – Скриншоты основных механик

7 Описание назначения и процесса загрузки проекта на Github

Первая функциональная загрузка проекта на GitHub, в которой добавлены почти все сцены, описаны пустые именованные функции. Добавлена доска - наложена колода из спрайтов 52 карты. Сделаны функции подбора и счета руки дилера и игрока. Сделана логика эйса/туза.

На рисунке 7 представлен последняя загрузка проекта на GitHub.

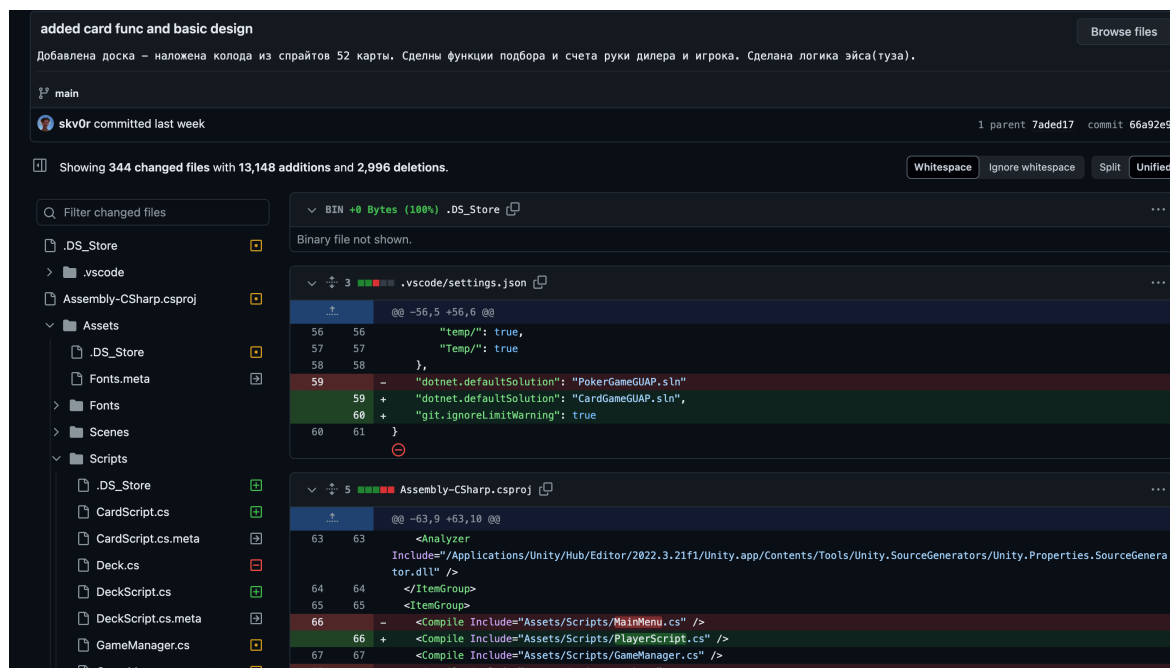


Рисунок 7 - Последняя загрузка проекта на GitHub

Вторая загрузка на GitHub: Исправлены логические ошибки, больше нет визуальных ошибок, связанных с выводом денег и невозможностью выхода во время игры. Обновлено главное меню: новая сцена настроек и правила с возможностью выхода. Добавлена музыка, которую легко включить или выключить в настройках.

На рисунке 8 представлен последняя загрузка проекта на GitHub.

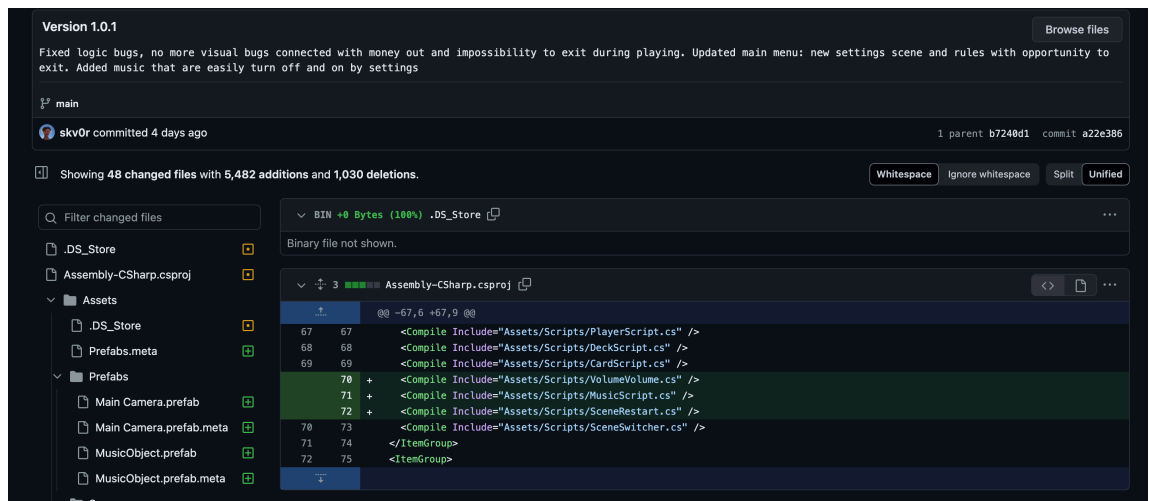


Рисунок 8 - Последняя загрузка проекта на GitHub

Вторая загрузка на GitHub: Новые спрайты, новые анимации для каждого меню и кнопок в игре. Новая система сбора карт, новая музыка, добавлены UnitTests, исправлен баг в Debug Mode связанный с отключением нужных сегментов и элементов.

На рисунке 9 представлен последняя загрузка проекта на GitHub.

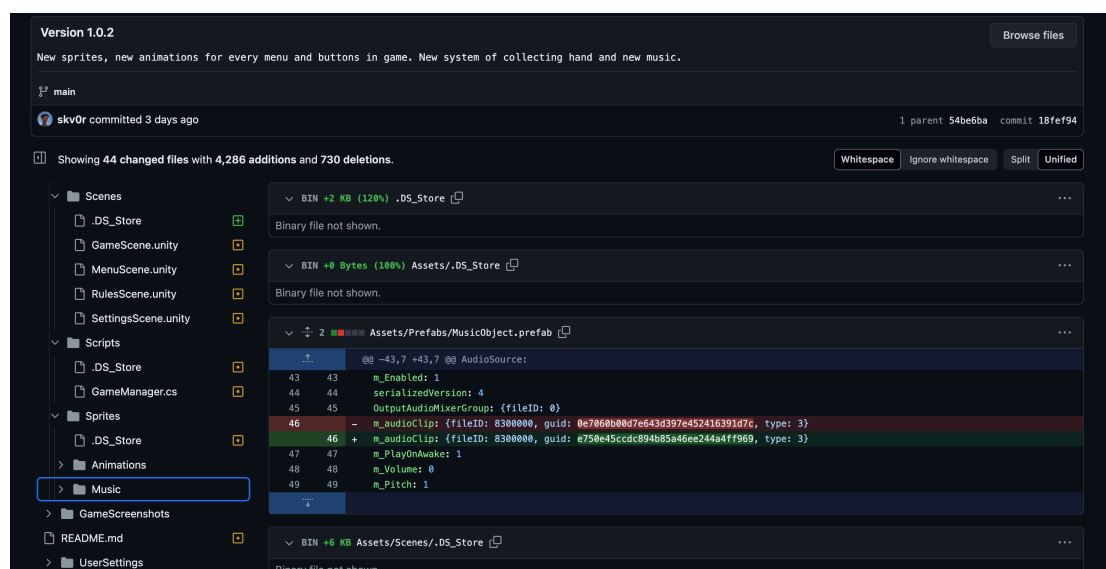


Рисунок 10 - Последняя загрузка проекта на GitHub

На рисунке 11-14 представлен файл README.md проекта на GitHub.

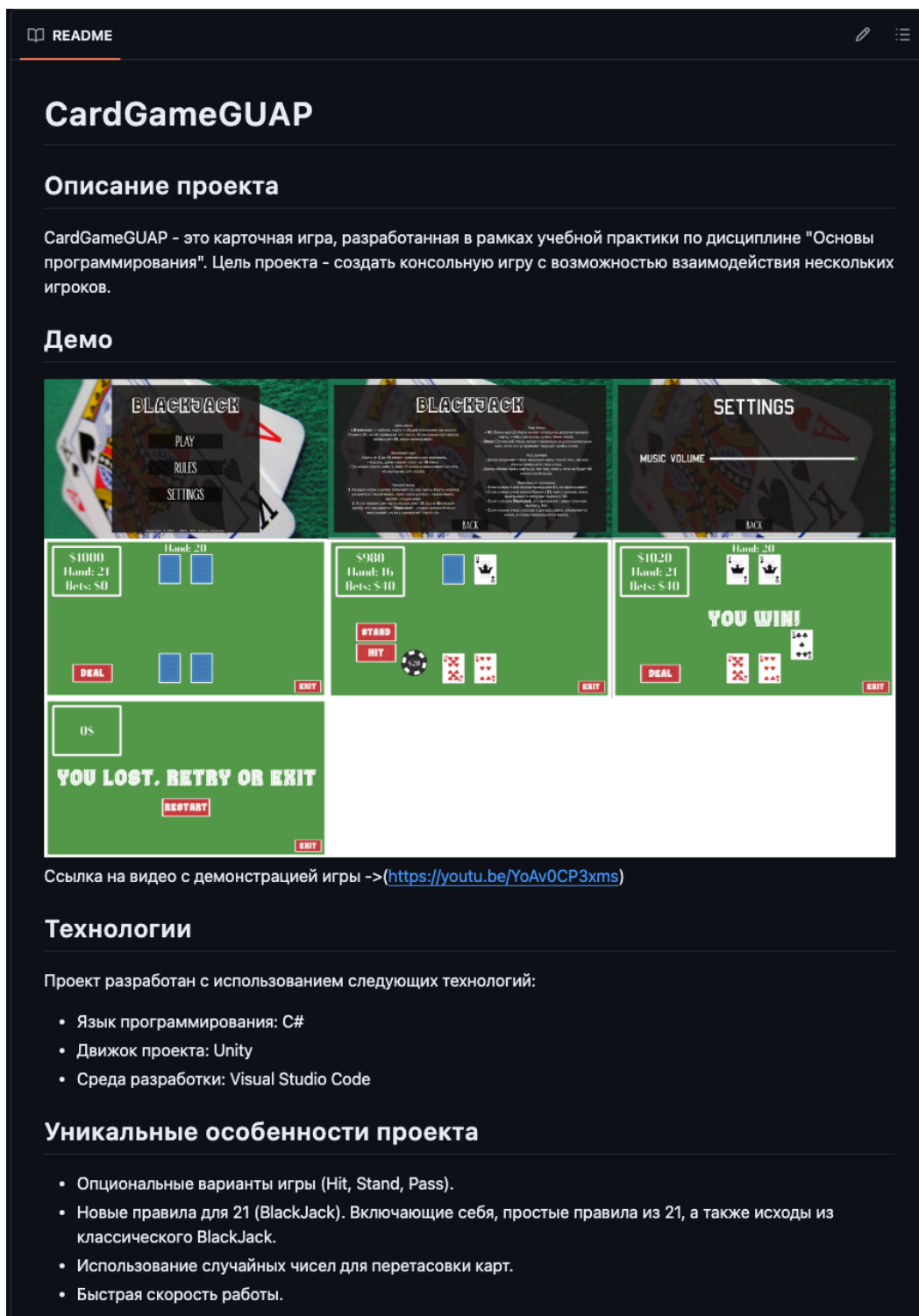


Рисунок 11 – Файл README.md.

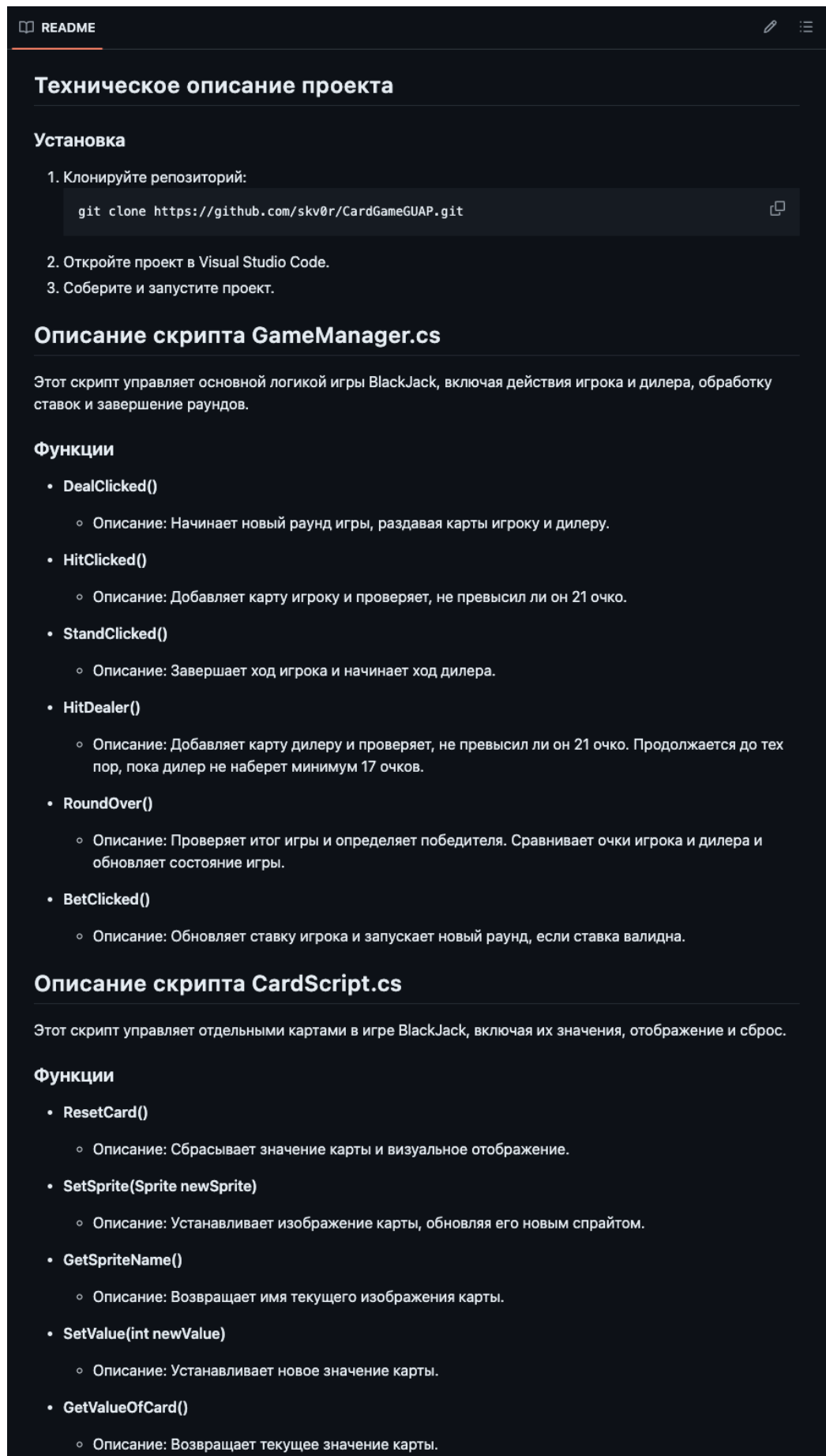


Рисунок 12 – Файл README.md.

Описание скрипта DeckScript.cs

Этот скрипт управляет колодой карт в игре BlackJack, включая их значения, перемешивание, раздачу и отображение рубашки карты.

Функции

- **GetCardValues()**
 - Описание: Возвращает массив значений всех карт в колоде.
- **Shuffle()**
 - Описание: Перемешивает карты в случайном порядке.
- **DealCard()**
 - Описание: Возвращает первую карту из колоды и удаляет её из списка.
- **GetCardBack()**
 - Описание: Возвращает спрайт рубашки карты.

Описание скрипта PlayerScript.cs

Этот скрипт управляет действиями игрока/дилера в игре BlackJack, включая начало раздачи, получение карт, проверку туза, корректировку денег и сброс руки.

Функции

- **StartHand()**
 - Описание: Раздает две карты и инициализирует руку игрока.
- **GetCard(GameObject card)**
 - Описание: Добавляет карту в руку игрока и обновляет его очки.
- **AceCheck()**
 - Описание: Корректирует значение туза, чтобы избежать превышения 21 очка.
- **AdjustMoney(int amount)**
 - Описание: Изменяет баланс игрока на указанную сумму.
- **ResetHand()**
 - Описание: Очищает руку и обнуляет очки игрока.

Вспомогательные файлы для сцен

Эти скрипты помогают управлять различными аспектами сцен в игре BlackJack, такими как музыка, перезагрузка сцен, переключение сцен и управление громкостью.

MusicScript.cs

Описание: Этот скрипт управляет воспроизведением фоновой музыки в игре.

Функции

- **PlayMusic()**
 - Описание: Начинает или возобновляет воспроизведение аудиотрека.
- **StopMusic()**
 - Описание: Останавливает текущий аудиотрек.
- **SetVolume(float volume)**
 - Описание: Устанавливает значение громкости и применяет его.

SceneRestart.cs

Описание: Этот скрипт отвечает за перезагрузку текущей сцены.

Функции

- **RestartScene()**
 - Описание: Перезагружает активную сцену.

SceneSwitcher.cs

Описание: Этот скрипт управляет переключением между сценами в игре.

Функции

- **SwitchToScene(string sceneName)**
 - Описание: Принимает имя сцены и загружает её.

VolumeVolume.cs

Описание: Этот скрипт управляет общим уровнем громкости в игре.

Функции

- **SetMasterVolume(float volume)**
 - Описание: Устанавливает общее значение громкости и применяет его.

🔗 Проблемы и решения

Проблемы

- Первоначально возникли сложности с реализацией алгоритма перемешивания карт.
- Были проблемы с поиском Sprites, многие шаблоны имели некачественные модели/ Маленькие размеры для большинства пользователей.
- Проблемы с неправильной работой анимаций на подобъектах, который находились на разных Layers in order, в следствии чего неправильно работающие алгоритмы и сцены.
- Проблемы с синхронизацией ходов и фишек в момент ставки и игры.

Решения

- Для перемешивания карт был использован алгоритм Фишера-Йетса.
- Множество моделей были отрисованы вручную, что позвонило удобнее компоновать сцены, сохраняя удобство пользователям.
- Для правильной работы алгоритмом и анимаций, они были разделены два уровня Canvas, что способствовало возможности установки подуровней Layers in order и удобной компоновке объектов.
- Синхронизация ходов была реализована с использованием флагов состояния.

Контакты

- Разработчик: Буренков Григорий
- Email: skvorgrand@gmail.com

Рисунок 14 – Файл README.md.

ЗАКЛЮЧЕНИЕ

В ходе выполнения учебной практики была разработана игра Blackjack на платформе Unity. Были изучены основы разработки игр, реализованы базовые механики и проведено тестирование программы. В процессе работы были применены различные технологии и инструменты, такие как C# и Unity, что позволило создать функциональное и удобное приложение. Особое внимание уделялось разработке пользовательского интерфейса, анимаций и игровой логики, что существенно повысило качество и удобство использования игры.

Проект был успешно завершен и размещен на GitHub, что позволяет легко демонстрировать его потенциальным пользователям и улучшать в будущем на основе отзывов и предложений. Размещение на GitHub также обеспечивает доступ к исходному коду, что способствует обмену знаниями и опытом с другими разработчиками.

Данный проект позволил закрепить теоретические знания на практике, развить навыки программирования и работы с игровыми движками, а также получить ценный опыт в управлении проектами и командной работе. Итоговый продукт представляет собой не только развлекательную, но и образовательную платформу, способную тренировать стратегическое мышление и навыки принятия решений у игроков.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. ГУАП, документация для учебного процесса. – URL: <https://guap.ru/regdocs/docs/uch> (дата обращения 10.05.2024)
2. Unity документация. – URL: <https://docs.unity3d.com/>
3. C# Programming Guide. – URL: <https://learn.microsoft.com/en-us/dotnet/csharp/>
4. Гергель, В.П. Современные языки и технологии параллельного программирования: Учебник / В.П. Гергель. — М.: МГУ, 2016. — 408 с.
5. YouTube канал Brackeys - один из самых популярных ресурсов для изучения по Unity – URL: <https://www.youtube.com/user/Brackeys>
6. Руководство по юнит-тестированию в Unity – URL: https://www.youtube.com/watch?v=PDYB32qAsLU&ab_channel=InfallibleCode
7. Руководство по анимациям в Unity – URL: <https://youtu.be/N7XQ9x5BDdU?si=r-J-bUllvtJCNTu>
8. Официальное руководство по Git и GitHub – URL: <https://git-scm.com/doc>

ПРИЛОЖЕНИЕ

Исходный код программы

CardScript.cs

```
using System.Collections;
using System.Collections.Generic;
using System.Linq.Expressions;
using UnityEngine;

public class CardScript : MonoBehaviour
{
    //Значения карт (буби,крести и тд)
    public int value = 0;

    public int GetValueOfCard()
    {
        return value;
    }

    public void SetValue(int newValue)
    {
        value = newValue;
    }

    public string GetSpriteName()
    {
        return GetComponent().sprite.name;
    }

    public void SetSprite(Sprite newSprite)
```

```

{
    gameObject.GetComponent<SpriteRenderer>().sprite = newSprite;
}

public void ResetCard()
{
    Sprite back =
GameObject.Find("Deck").GetComponent<DeckScript>().GetCardBack();
    gameObject.GetComponent<SpriteRenderer>().sprite = back;
    value = 0;
}
}

```

DeckScript.cs

```
using System.Collections;
using System.Collections.Generic;
using System.Globalization;
using UnityEngine;

public class DeckScript : MonoBehaviour
{
    public Sprite[] cardSprites;
    int[] cardValues = new int[53];
    int currentIndex = 0;

    void Start()
    {
        GetCardValues();
    }

    void GetCardValues()
    {
        int num = 0;
        // Цикл для присвоения значений картам
        for (int i = 0; i < cardSprites.Length; i++)
        {
            num = i;
            // Подсчит количество карт, 52
            num %= 13;
            // если после x/13 есть остаток, то остаток присвоится остаток
            // иначе присвоиться 10
            if(num > 10 || num == 0)
            {
```

```

        num = 10;
    }
    cardValues[i] = num++;
}
}

```

```

public void Shuffle()
{
    // метод обмена данными массива
    for(int i = cardSprites.Length - 1; i > 0; --i)
    {
        int j = Mathf.FloorToInt(Random.Range(0.0f, 1.0f) * cardSprites.Length -
1) + 1;
        Sprite face = cardSprites[i];
        cardSprites[i] = cardSprites[j];
        cardSprites[j] = face;

        int value = cardValues[i];
        cardValues[i] = cardValues[j];
        cardValues[j] = value;
    }
    currentIndex = 1;
}

```

```

public int DealCard(CardScript cardScript)
{
    cardScript.SetSprite(cardSprites[currentIndex]);
    cardScript.SetValue(cardValues[currentIndex]);
    currentIndex++;
    return cardScript.GetValueOfCard();
}

```

```
}  
  
public Sprite GetCardBack()  
{  
    return cardSprites[0];  
}  
}
```

GameManager.cs

```
using System;
using System.Collections;
using System.Collections.Generic;
using UnityEngine;
using UnityEngine.SocialPlatforms.Impl;
using UnityEngine.UI;

public class GameManager : MonoBehaviour
{
    // Игровые кнопки
    public Button dealBtn;
    public Button hitBtn;
    public Button standBtn;
    public Button betBtn;
    public Button restartBtn;

    private int standClicks = 0;

    // Доступ к скриптам дилера и игрока
    public PlayerScript playerScript;
    public PlayerScript dealerScript;

    // Обще-доступный текст для худа
    public Text scoreText;
    public Text dealerScoreText;
    public Text betsText;
    public Text cashText;
    public Text mainText;
    public Text standBtnText;
```

```

// Вторая карта дилера (скрытая)
public GameObject hideCard;

// Ставка по умолчанию
int pot = 0;

void Start()
{
    // Наложение на объект click способности
    dealBtn.onClick.AddListener(() => DealClicked());
    hitBtn.onClick.AddListener(() => HitClicked());
    standBtn.onClick.AddListener(() => StandClicked());
    betBtn.onClick.AddListener(() => BetClicked());

    //дефолтная настройка сцены при запуске
    hitBtn.gameObject.SetActive(false);
    standBtn.gameObject.SetActive(false);
    betBtn.gameObject.SetActive(false);
    restartBtn.gameObject.SetActive(false);
}

private void DealClicked()
{
    //проверка на проигрыш в игре (0 денег)
    if (playerScript.GetMoney() < 20)
    {
        restartBtn.gameObject.SetActive(true);
        hitBtn.gameObject.SetActive(false);
        standBtn.gameObject.SetActive(false);
        betBtn.gameObject.SetActive(false);
    }
}

```

```

dealBtn.gameObject.SetActive(false);
playerScript.ResetHand();
dealerScript.ResetHand();
cashText.text = "";
scoreText.text = "0$";
betsText.text = "";
dealerScoreText.text = "";
mainText.text = "You lost. RETRY or EXIT";
} else
{
    // Сброс раунда, скрывает временный текст, подготавливает руку
    playerScript.ResetHand();
    dealerScript.ResetHand();
    // Скрывает счет руки на начало игры
    betBtn.gameObject.SetActive(true);
    dealerScoreText.gameObject.SetActive(false);
    mainText.gameObject.SetActive(false);
    dealerScoreText.gameObject.SetActive(false);
    GameObject.Find("Deck").GetComponent<DeckScript>().Shuffle();
    playerScript.StartHand();
    dealerScript.StartHand();
    // Обновление руки на табло
    scoreText.text = "Hand: " + playerScript.handValue.ToString();
    dealerScoreText.text = "Hand: " + dealerScript.handValue.ToString();
    // Раскрывает руку дилера (скрытую карту)
    hideCard.GetComponent<Renderer>().enabled = true;
    // Кнопки управления видимости
    dealBtn.gameObject.SetActive(false);
    hitBtn.gameObject.SetActive(true);
    standBtn.gameObject.SetActive(true);

```

```

        standBtnText.text = "Stand";
        // Установка стандартной ставки
        pot = 40;
        betsText.text = "Bets: $" + pot.ToString();
        playerScript.AdjustMoney(-20);
        cashText.text = "$" + playerScript.GetMoney().ToString();
    }

}

private void HitClicked()
{
    // Проверка карт на столе (их нахождение)
    if (playerScript.cardIndex <= 10)
    {
        playerScript.GetCard();
        scoreText.text = "Hand: " + playerScript.handValue.ToString();
        if (playerScript.handValue > 20) RoundOver();
    }
}

private void StandClicked()
{
    standClicks++;
    if (standClicks > 1) RoundOver();
    HitDealer();
    standBtnText.text = "Call";
}

private void HitDealer()

```

```

{
    while (dealerScript.handValue < 16 && dealerScript.cardIndex < 10)
    {
        dealerScript.GetCard();
        dealerScoreText.text = "Hand: " + dealerScript.handValue.ToString();
        if (dealerScript.handValue > 20) RoundOver();
    }
}

// Проверка победы/ поражения, перебор руки
void RoundOver()
{
    betBtn.gameObject.SetActive(false);
    mainText.gameObject.SetActive(true);
    // Булевы значения рук дилера и игрока (проверка на blackjack)
    bool playerBust = playerScript.handValue > 21;
    bool dealerBust = dealerScript.handValue > 21;
    bool player21 = playerScript.handValue == 21;
    bool dealer21 = dealerScript.handValue == 21;

    // Если stand нажат дважды, нет блекджека или перебора, выход из
функции
    if (standClicks < 2 && !playerBust && !dealerBust && !player21 &&
!dealer21) return;

    bool roundOver = true;
    // У обоих перебор, возврат ставок
    if (playerBust && dealerBust)
    {

        mainText.text = "All Bust: Bets returned";
        playerScript.AdjustMoney(pot / 2);

```

```

    }

    // У игрока перебор, у дилера нет, или если дилер имеет руку больше,
ПОБЕДА ДИЛЕРА
    else if (playerBust || (!dealerBust && dealerScript.handValue >
playerScript.handValue))
    {
        mainText.text = "Dealer wins!";
    }

    // перебор дилера, у игрока нет, или если игрок имеет руку больше,
ПОБЕДА ИГРОКА
    else if (dealerBust || playerScript.handValue > dealerScript.handValue)
    {
        mainText.text = "You win!";
        playerScript.AdjustMoney(pot);
    }

    //Проверка на ничью, возврат ставок
    else if (playerScript.handValue == dealerScript.handValue)
    {
        mainText.text = "Push: Bets returned";
        playerScript.AdjustMoney(pot / 2);
    }
    else
    {
        roundOver = false;
    }

    // Установка ui для следующей партии
    if (roundOver)
    {
        hitBtn.gameObject.SetActive(false);

```

```

        standBtn.gameObject.SetActive(false);
        dealBtn.gameObject.SetActive(true);
        mainText.gameObject.SetActive(true);
        dealerScoreText.gameObject.SetActive(true);
        hideCard.GetComponent<Renderer>().enabled = false;
        cashText.text = "$" + playerScript.GetMoney().ToString();
        standClicks = 0;
    }
}
// Добавление денег ставки по нажатию
void BetClicked()
{
    if (playerScript.GetMoney() < 20)
    {
        betBtn.gameObject.SetActive(false);
        mainText.text = "No money";
        mainText.gameObject.SetActive(true);
    } else
    {
        Text newBet = betBtn.GetComponentInChildren(typeof(Text)) as Text;
        int intBet = int.Parse(newBet.text.ToString().Remove(0, 1));
        playerScript.AdjustMoney(-intBet);
        cashText.text = "$" + playerScript.GetMoney().ToString();
        pot += (intBet * 2);
        betsText.text = "Bets: $" + pot.ToString();
    }
}
}
}

```

SceneRestart.cs

```
using UnityEngine;
using UnityEngine.SceneManagement;

public class SceneRestart : MonoBehaviour
{
    // Метод для перезапуска текущей сцены
    public void RestartScene()
    {
        // Получаем индекс текущей сцены
        int currentSceneIndex = SceneManager.GetActiveScene().buildIndex;

        // Перегружаем текущую сцену по ее индексу
        SceneManager.LoadScene(currentSceneIndex);
    }
}
```

MusicScript.cs

```
using System.Collections;
using System.Collections.Generic;
using UnityEngine;

public class MusicScript : MonoBehaviour
{
    public GameObject MusicObject;
    private AudioSource audioSrc1;
    public GameObject[] objs11;

    void Awake()
    {
        objs11 = GameObject.FindGameObjectsWithTag("Sound");
        if (objs11.Length == 0)
        {
            MusicObject = Instantiate(MusicObject);
            MusicObject.name = "MusicObject";
            DontDestroyOnLoad(MusicObject.gameObject);
        }
        else
        {
            MusicObject = GameObject.Find("MusicObject");
        }
    }

    void Start()
    {
        audioSrc1 = MusicObject.GetComponent<AudioSource>();
    }
}
```

VolumeVolume.cs

```
using System.Collections;
using System.Collections.Generic;
using UnityEngine;

public class VolumeVolume : MonoBehaviour
{
    private AudioSource audioScr;
    private float musicVolume = 1f;

    void Start()
    {
        audioScr = GetComponent<AudioSource>();
    }

    void Update()
    {
        audioScr.volume = musicVolume;
    }

    public void SetVolume (float vol)
    {
        musicVolume = vol;
    }
}
```

PlayerScript.cs

```
using System.Collections;
using System.Collections.Generic;
using UnityEngine;

//PlayerScript это функционал игрока и дилера (обоих), получение денег,
ставка денег, получение карт, подсчет тузов как 1 или 11
public class PlayerScript : MonoBehaviour
{

    // Получение скриптов карт и стола
    public CardScript cardScript;
    public DeckScript deckScript;

    // Сумма колоды (руки) у игрока/дилера
    public int handValue = 0;

    // Стартовое значение денег при запуске игры
    private int money = 1000;

    // Массив карт на столе
    public GameObject[] hand;

    // Индекс следующей карты для игры
    public int cardIndex = 0;

    // Остлеживание туза от 1 до 11
    List<CardScript> aceList = new List<CardScript>();

    public void StartHand()
    {
        GetCard();
    }
}
```

```

    GetCard();
}

// Добавление значения к руке дилера/игрока
public int GetCard()
{
    // Получение значения карты, чтобы использовать на столе
    int cardValue =
deckScript.DealCard(hand[cardIndex].GetComponent<CardScript>());
    // Показ карт столе
    hand[cardIndex].GetComponent<Renderer>().enabled = true;
    // Добавление карты в тотал рук
    handValue += cardValue;
    // Если значение 1, то туз 11
    if(cardValue == 1)
    {
        aceList.Add(hand[cardIndex].GetComponent<CardScript>());
    }
    // Если значение 11, то присвоится стандартное значение 1
    AceCheck();
    cardIndex++;
    return handValue;
}

// Поиск нужного значения туза (1 или 11)
public void AceCheck()
{
    // for each ace in the lsit check
    foreach (CardScript ace in aceList)
    {

```

```

    if(handValue + 10 < 22 && ace.GetValueOfCard() == 1)
    {
        // если доходит, то присваивается card object value и hand
        ace.SetValue(11);
        handValue += 10;
    } else if (handValue > 21 && ace.GetValueOfCard() == 11)
    {
        // если доходит, то присваивается gameobject value и hand value
        ace.SetValue(1);
        handValue -= 10;
    }
}
}
}

```

// Добавление денег на баланс для ставок

```

public void AdjustMoney(int amount)
{
    money += amount;
}

```

// Вывод пользователю количество денег в данный момент

```

public int GetMoney()
{
    return money;
}

```

// Скрывает все карты, сбрасывает значения блоков рук и всех
вспомогательных функций

```

public void ResetHand()
{

```

```
for(int i = 0; i < hand.Length; i++)
{
    hand[i].GetComponent<CardScript>().ResetCard();
    hand[i].GetComponent<Renderer>().enabled = false;
}
cardIndex = 0;
handValue = 0;
aceList = new List<CardScript>();
}
}
```